

Lyric Clip Generator

How this app was built

A build-and-decisions log of the work that took this app from an empty repo to a live, paying product — written for the whole team.

Version 1.0 — MVP complete

Repository: github.com/SevrinaLee/lyric-clip-generator

Live: lyric-clip-generator.vercel.app

Prepared: 12 July 2026

Built by SevrinaLee with Claude Code (pair-built).

1. What this app is

Lyric Clip Generator turns a piece of audio into short, vertical, platform-ready lyric video clips. A user uploads a song (or cover, poem, or podcast), adds the lyrics, and the app finds the most hook-worthy moments, scores them, lets the user pick a visual template and preview it, then renders a finished MP4 with burned-in captions ready to post to TikTok, Reels, or Shorts.

It is a full product, not a prototype: real accounts, real per-user data isolation, real video rendering, and real payments (card, PayNow, GrabPay) in Singapore dollars.

The one-line pitch

Audio in - three scroll-stopping, hook-scored lyric clips out, in minutes, with no editing skills required.

How to read this document

Section 2 tells the story in the order it happened, grouped into the phases we actually worked in, with the reasoning behind the important decisions. At the end of each phase you will see the user-journey status as it stood at that moment, so you can watch the app grow. Section 3 is the current status. Appendix A is a plain chronological list of every commit. Appendix B is a technical reference for engineers.

Throughout, each step of the journey is status-coded:

- Live - built, deployed, and working in production.
- Built, needs config - the code is done, but a key or setting is required before it fully works.
- Verify on device - built, but not yet visually confirmed at that screen size.
- Not built yet - does not exist at this stage.

2. The build story, phase by phase

Phase 1 - The core engine (Sprints 1-5)

The first push built the entire spine of the product from an empty repository, following a plan pack (PRD, architecture, sprint breakdown) that shipped with the template. Five sprints landed back to back:

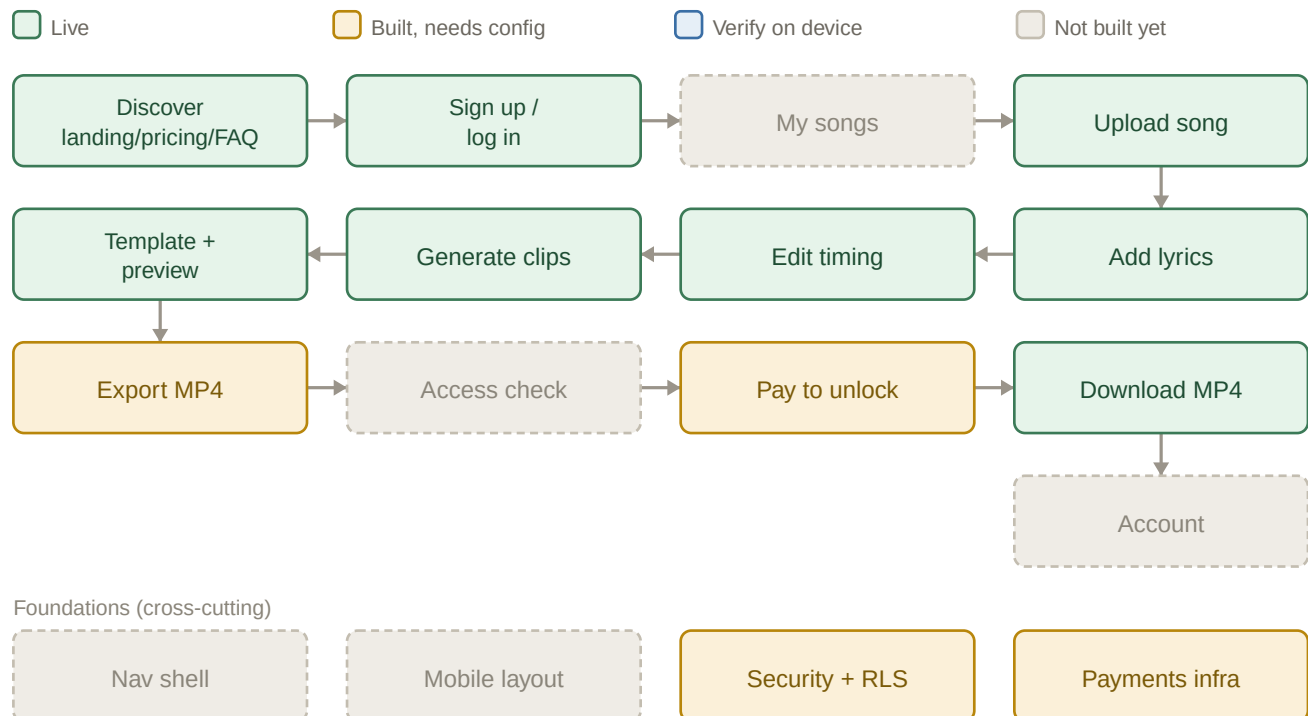
- Sprint 1 - database, song upload, and lyric entry. Audio is stored in Supabase Storage; songs and lyrics live in Postgres.
- Sprint 2 - the heart of the product: rule-based hook scoring (which picks the catchiest moments without needing an AI key), a template picker, and real MP4 export via ffmpeg.
- Sprint 3 - a Stripe checkout payment gate so exports could be monetised.
- Sprint 4 - optional auto-transcription via OpenAI Whisper, for users who do not want to type lyrics.
- Sprint 5 - accounts and a per-user security lock-down using Postgres Row-Level Security (RLS), so one user can never see another's data.

KEY DECISION - WHY RULE-BASED SCORING, NOT AI

The 'intelligence' that picks hook moments is deliberately rule-based, so the core product works with zero AI API keys and zero per-use cost. AI transcription is an optional add-on, never a dependency. This keeps the app cheap to run and resilient.

KEY DECISION - SECURITY IS ENFORCED BY THE DATABASE, NOT THE APP

Data isolation is enforced by Postgres Row-Level Security, so even a bug in application code cannot leak one user's songs or payments to another. The powerful service-role key is only ever used server-to-server, never in the browser.



Journey after Phase 1: the whole create-to-download loop exists. Export is amber because it worked locally but had not yet survived the production environment. Payments are a scaffold with no live key.

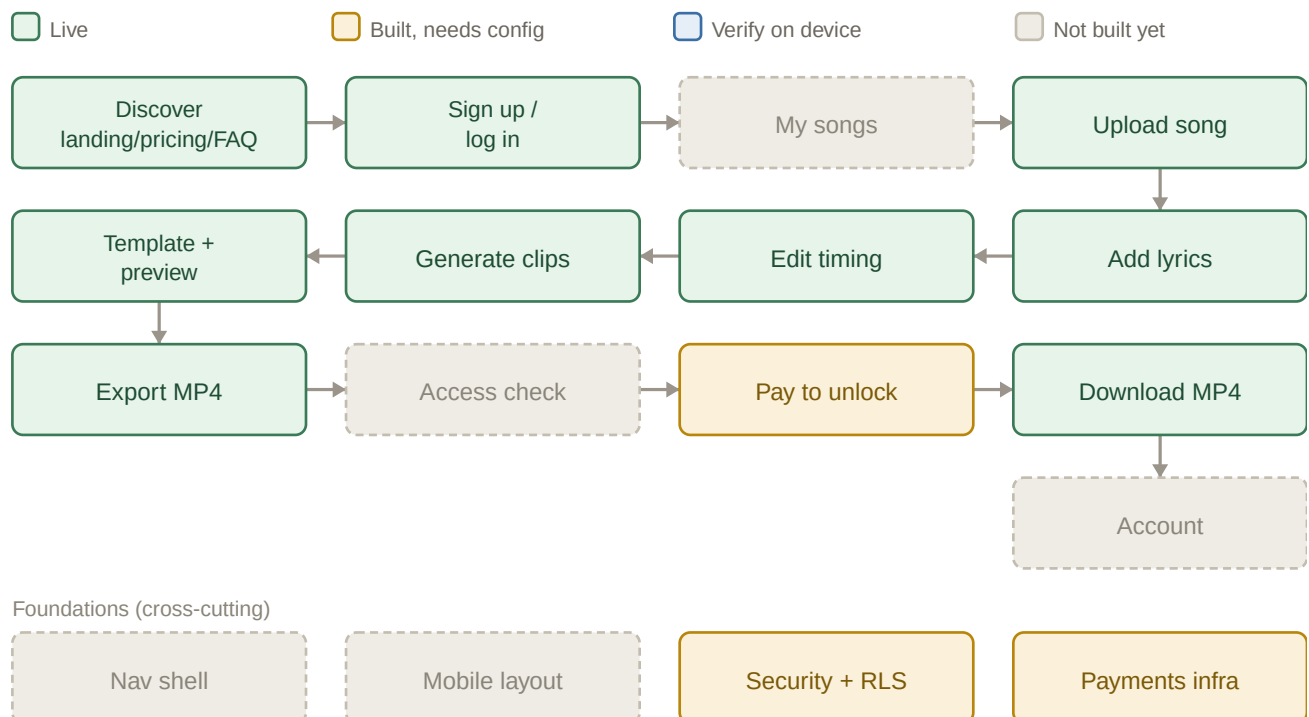
Phase 2 - Making it actually work in production, plus a UX pass

A working demo on a laptop is not a working product. Several things that passed locally broke on the live Vercel/serverless environment, and fixing them was a real chunk of the work — the kind of thing worth documenting because it will recur on any similar project.

- The MP4 export crashed three separate times in production for three different reasons, each invisible on the local machine.
- The template picker, lyric editing, and clip preview were rebuilt to be genuinely usable: edit lyrics with or without timestamps, a live template preview, and a tap-to-time tool that lets you tap along to the song instead of typing timecodes.
- A full brand redesign gave every page a consistent, considered look.

THE PRODUCTION EXPORT SAGA (A CAUTIONARY TALE)

1. The font file path resolved to a numeric module id inside the production bundle, not a real path - crash. Fixed by shipping the font with the app.
 2. The ffmpeg binary was missing from the serverless bundle, and a stale build cache hid it - crash. Fixed by forcing the binary into the bundle.
 3. The production ffmpeg build simply did not include the 'drawtext' feature we relied on - crash. Fixed by switching caption rendering to the 'ass' subtitle method the binary did support.
- Lesson: 'works on my machine' is not 'works in production' - each was only found by reading real production logs.



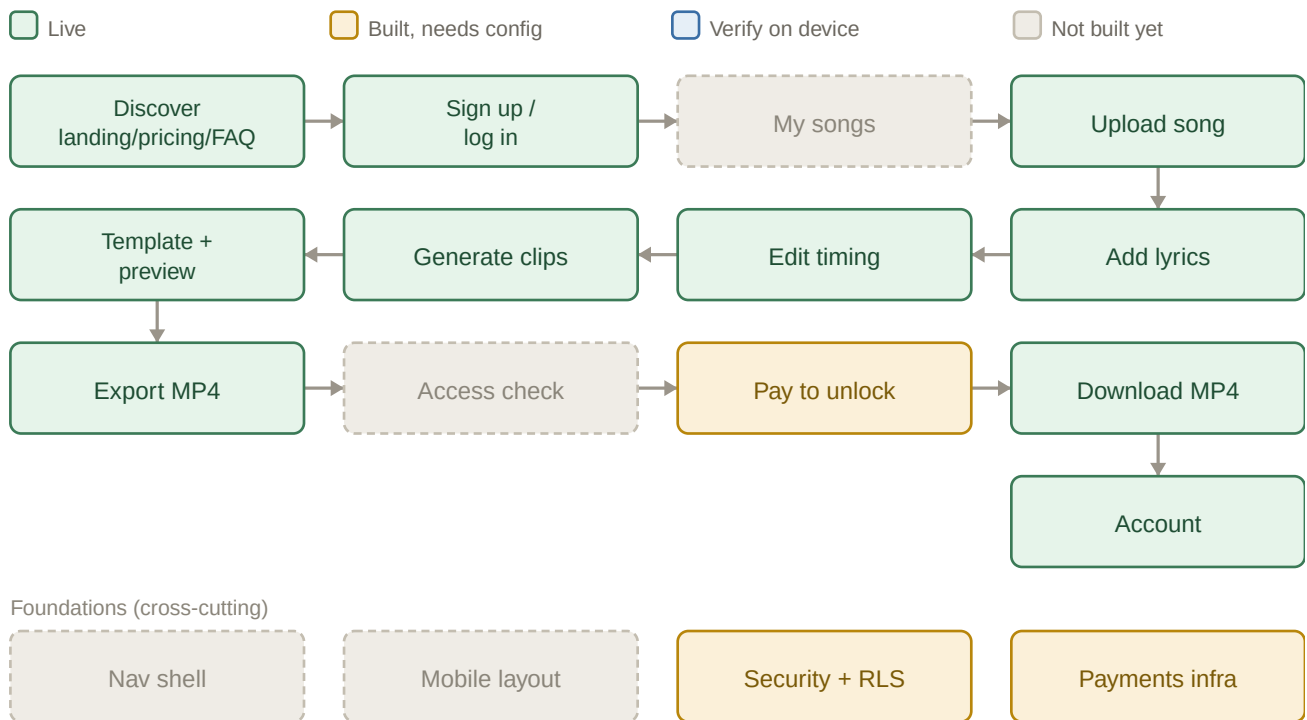
Journey after Phase 2: export is now genuinely live in production. The account area does not exist yet.

Phase 3 - Growth features

With the core solid, the next phase added the things a real product needs around the edges: a forgot-password flow, dedicated Pricing and FAQ pages, more visual templates (including animated gradient backgrounds while keeping captions readable), an account page where users can set a display name and manage billing, and the payments code upgraded to offer multiple methods (card, PayNow QR, GrabPay) in Singapore dollars.

KEY DECISION - PAYMENTS BUILT MODE-AGNOSTIC

Checkout does not hard-code which payment methods to show. It lets the Stripe dashboard decide, so enabling PayNow or GrabPay is a settings change, not a code change. The currency was set to SGD because PayNow and GrabPay require it.



Journey after Phase 3: the account area is live. Payments remain amber - the code is complete, but no live Stripe key was connected yet.

Phase 4 - Security you can prove, and a smarter access model

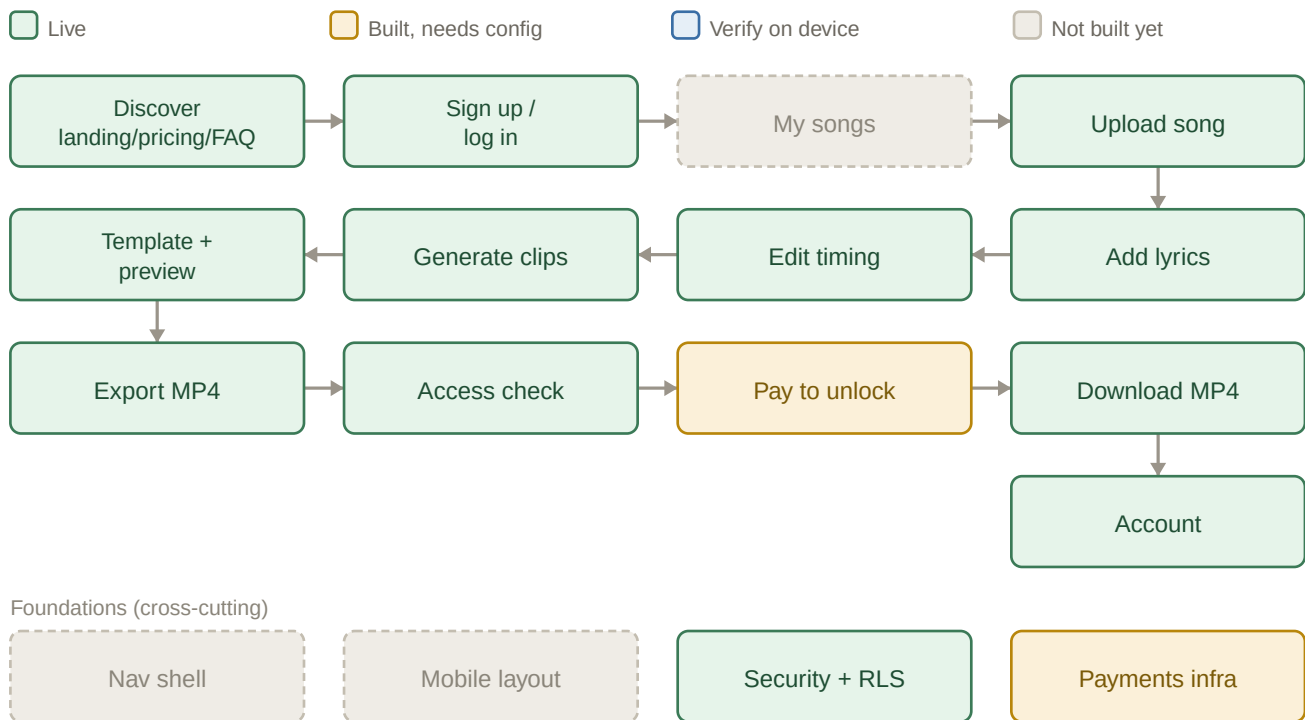
The project's own rules require that any change touching data or auth be backed by automated security tests. This phase delivered that: an executable DevSecOps test suite covering the four things that matter, wired to run automatically on every code change.

- Data isolation - proves one user cannot read or modify another's songs, payments, or profile.
- SQL-injection - fires real attack strings at every input and proves they are treated as harmless text.
- Brute-force - proves rapid failed logins get rate-limited.
- Data-exfiltration - proves list views never leak other users' rows or sensitive fields.

It also introduced a smarter download-access model: founder/QA accounts that get everything free, and one free song for every new user (their first download), with everything after that paid. Both are enforced server-side and locked so a user cannot promote themselves.

KEY DECISION - FOUNDER ACCESS AND THE FREE SONG ARE LOCKED AT THE DATABASE LEVEL

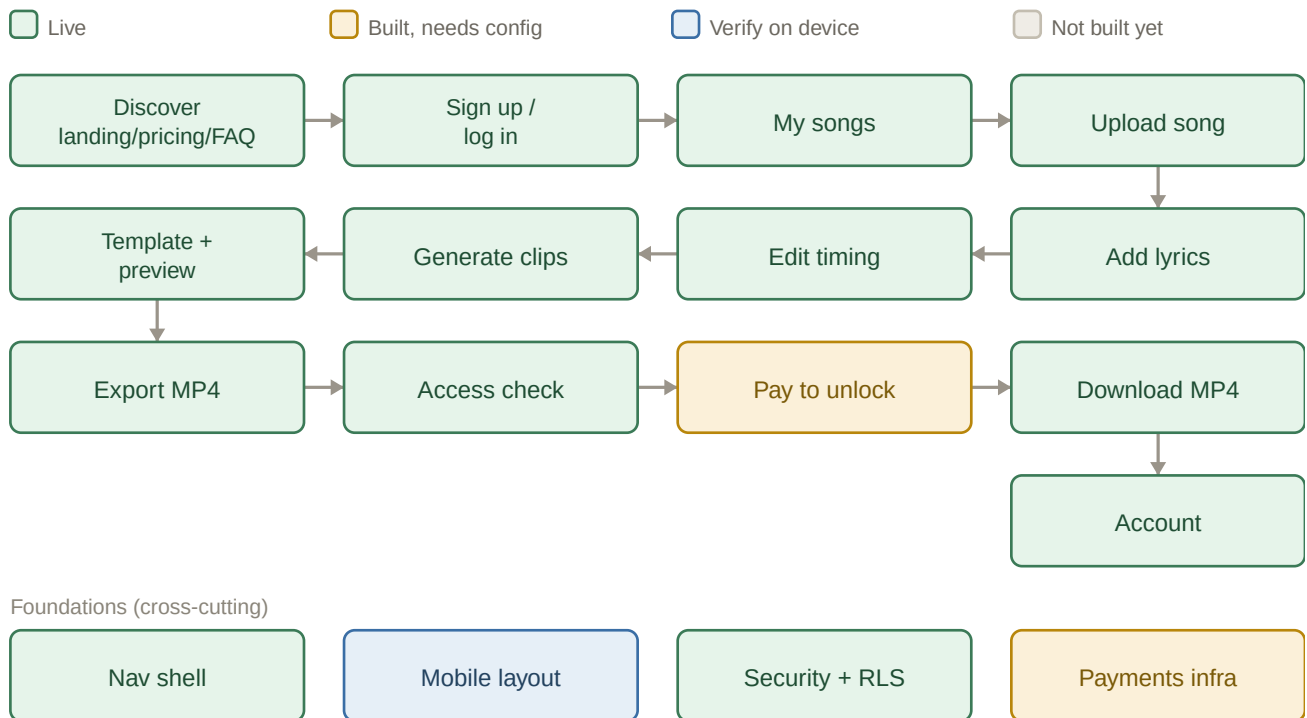
The 'is this account a founder' and 'which song is free' flags can only be written by trusted server code. The database revokes a normal user's ability to change them, so no one can grant themselves free access. A security test guards this forever.



Journey after Phase 4: the access check is live (founder / first-song-free / paid), and security is verified by an automated suite running in CI.

Phase 5 - Navigation shell and mobile

Until now the app had a single top bar. This phase turned it into a proper responsive shell: a left sidebar on desktop that collapses into a hamburger-drawer menu on phones, shown only to signed-in users so the landing page stays welcoming. A new 'My songs' page gave signed-in users a home base listing their own uploads. A follow-up pass tightened spacing, tap targets, and the lyrics editor for small screens.

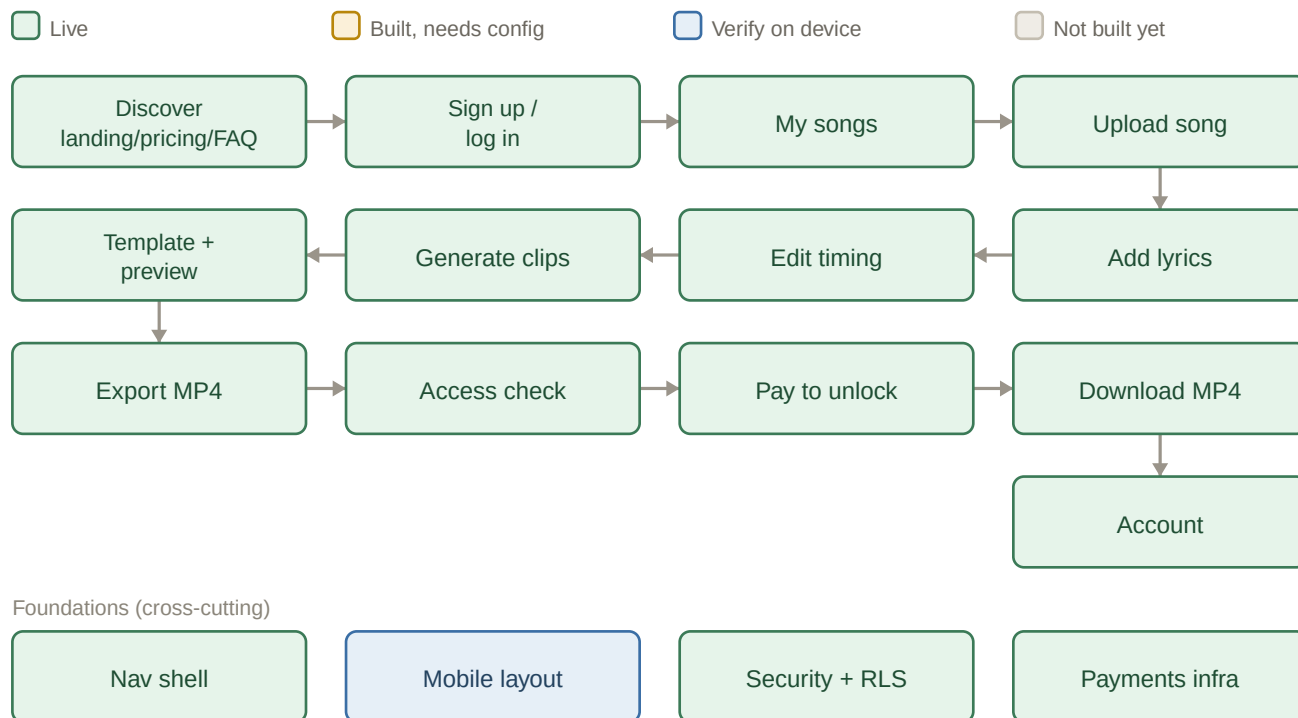


Journey after Phase 5: My songs and the navigation shell are live. Mobile layout is blue - built and code-verified, but the phone-width view had not been driven on a real device in the build environment.

Phase 6 - Payments go live

The final step to a true 1.0: connecting a live Singapore Stripe account. The secret key and webhook were configured, and the whole payment path was verified end to end without spending real money — the live key

was confirmed valid and in live mode, a real checkout session was created and shown to offer card, GrabPay, PayNow and Link, and the webhook was proven to accept correctly-signed events and reject forged ones.



Journey at v1.0: every step of the core loop is live, including real multi-method payments. Only the on-device mobile spot-check remains as blue.

3. Where the app stands today (v1.0)

Every step of the core user journey works in live production: a real person can discover the app, sign up, upload audio, generate and preview clips, pay by card / PayNow / GrabPay, and download a finished MP4. That is the definition of a shippable 1.0.

Live and working

Discovery pages, accounts and password reset, My songs, upload, lyric entry and timing editors, rule-based clip generation, templates and preview, real MP4 export, the access gate (founder / first-song-free / paid), real multi-method payments, the account area, the navigation shell, and an automated security suite running on every change.

Small open items (not blockers)

- Auto-transcribe needs an OpenAI key to switch on. It is the optional path; typing or pasting lyrics works fully without it.
- Mobile layout is built and code-verified but should get a 60-second look on a real phone (the build environment could not shrink below desktop width).
- Two housekeeping items parked by choice: a second founder email still needs to sign up before it can be flagged, and a founder test account is on a temporary password.

THE SINGLE HIGHEST-VALUE NEXT STEP IS DONE

At the time the journey was first mapped, the biggest gap between 'great demo' and 'can take money' was the Stripe key. That is now connected and verified - the app can accept real payments.

Appendix A. Chronological commit log

Every commit, oldest to newest. Read top-to-bottom, this is the same story as Section 2 at finer grain.

Date	What changed and why
2026-06-24	Initialise project, plan pack (PRD, architecture, sprints), initial schema migration
2026-07-08	Restore plan pack + migrations, add storage-buckets migration
2026-07-09	Sprint 1 - demo gallery, song upload, lyric entry
2026-07-09	Sprint 2 - hook scoring, template picker, real MP4 export
2026-07-09	Sprint 3 - Stripe checkout payment gate
2026-07-09	Sprint 4 - Whisper auto-transcription
2026-07-09	Sprint 5 - auth + per-user RLS lock-down
2026-07-09	Fix auth production bugs (site URL, rate limit, hide unconfigured Google button)
2026-07-09	Hide export/edit controls on songs the viewer does not own
2026-07-09	Brand redesign across all pages
2026-07-09	Editable lyrics without timestamps, live template preview, synced clip preview
2026-07-09	Delete lyric lines, correctable timing for all songs
2026-07-09	Tap-to-time lyrics, broader hero copy, account/billing nav
2026-07-09	Fix export crash in production (ERR_INVALID_ARG_TYPE - font path)
2026-07-09	Fix export crash (ffmpeg ENOENT - binary missing from bundle)
2026-07-09	Fix export failing ('Filter not found' - switch to ass subtitles)
2026-07-09	Forgot password, FAQ + pricing pages, gradient background templates
2026-07-09	Multi-method Stripe checkout (SGD/PayNow-ready) + profile display name
2026-07-10	Add DevSecOps security suite (tests/security/)
2026-07-11	Sanitise env secrets (BOM/whitespace) + preflight in security tests
2026-07-11	Founder comp access + one free song per user
2026-07-11	Responsive nav shell + My songs page (Phase 1)
2026-07-11	Mobile optimisation pass (Phase 2)
2026-07-12	Connect and verify live Stripe payments (v1.0)

Appendix B. Technical reference (for engineers)

Stack

- Next.js 15 (App Router), React 19, TypeScript, Tailwind v4 - hosted on Vercel.
- Supabase - Postgres, Auth, and Storage; sessions via cookies (@supabase/ssr).
- Stripe - Checkout + webhooks, SGD, dashboard-controlled payment methods.
- ffmpeg (via ffmpeg-static) - trims audio and burns in captions using the libass 'ass' subtitle filter; a bundled Noto Sans font is shipped with the app.

How data isolation works

Every user-owned table (songs, lyrics, clip_segments, payments, exports, profiles) has Row-Level Security enabled with an `auth.uid() = user_id` policy. Adding a new user-scoped table requires an accompanying RLS migration in the same change. The service-role key bypasses RLS and is used only server-to-server (the Stripe webhook and the free-song claim), never in a request carrying a user session.

The download access model

Centralised in `lib/access.ts` and enforced in the export-download route. A clip is downloadable free when the account is a founder (`is_founder = true`), or it is the user's one claimed free song (`profiles.free_song_id`, set once by trusted server code), or a paid payment exists for the song. The `is_founder` and `free_song_id` columns are locked at the database grant level so a user cannot self-promote or rotate their free song.

The security suite

`tests/security/` runs on every push/PR via GitHub Actions. It covers data isolation, SQL-injection prevention, brute-force rate-limiting, data-exfiltration prevention, and privilege-escalation prevention (17 tests). It runs against the real Supabase project with disposable users that are cleaned up after each run.

Notable production lessons

- Serverless bundles differ from local: file paths, native binaries, and build caches all bit us on the ffmpeg export path.
- The prebuilt ffmpeg-static binary lacks the drawtext filter; libass ('ass') is the reliable way to burn captions.
- Supabase's service-role key in this project carried a byte-order mark; env parsing must strip it or JWTs silently fail.

Regenerating this document

This PDF and its Markdown twin are generated from one source, `docs/build-doc/generate.mjs`. To document a new phase: add a milestone status map and the new content blocks there, then run ``npm install`` (first time) and ``node generate.mjs`` from `docs/build-doc`. Both files regenerate in sync.